

ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
РАДІОЕЛЕКТРОНІКИ
КАФЕДРА ЕОМ

**КУРСОВИЙ ПРОЕКТ:
ПРОГРАМУВАННЯ**

на тему: "Розроблення програмного продукту для роботи з файловою базою
даних про прихід товарів на склад підприємства"

Студента 1 курсу КІУКІу-25-1 групи
Спеціальності F7 "Комп'ютерна інженерія"
першого (бакалаврського) рівня
Лучанінова П. О.

Національна шкала _____
Кількість балів: _____ Оцінка: ECTS _____

Члени комісії

_____	_____
(підпис)	(прізвище та ініціали)
_____	_____
(підпис)	(прізвище та ініціали)
_____	_____
(підпис)	(прізвище та ініціали)

м. Харків – 202_ рік

ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Факультет КІУ

Кафедра ЕОМ

ЗАВДАННЯ

на курсовий проект: Програмування
студенту 1-го курсу групи КІУКІу-25-1
Лучанінову Петру Олександровичу

1. Тема проекту: " Розроблення програмного продукту для роботи з файловою базою даних про прихід товарів на склад підприємства".
2. Термін здачі студентом закінченого проекту " __ " _____ 202_ р.
3. Вхідні дані до проекту: літературні джерела, технічна документація щодо розроблення програм, ДСТУ з оформлення документації.
4. Зміст пояснювальної записки: Вступ. Специфікація проекту. Програмна документація. Висновки. Додатки.
5. Перелік графічного матеріалу: UML-діаграма класів програмної системи.

**Календарний план
виконання курсового проекту**

№ з/п	Назва етапу роботи	Строк виконання за планом	Відмітка про виконання
1.	З'ясування загальної постановки завдання. Розроблення чернетки першого розділу пояснювальної записки		
2.	Деталізація завдань курсового проектування. Уточнення архітектури програмної системи. Розроблення остаточного варіанту першого розділу пояснювальної записки		
3.	Програмна реалізація, налагодження та тестування застосунку		
4.	Остаточне налагодження та тестування застосунку. Розроблення чернетки другого розділу пояснювальної записки, висновків, списку використаних джерел, додатків, вступу, реферату)		
5.	Розроблення остаточного варіанта пояснювальної записки та підготовка електронної презентації		

Дата видачі завдання "___" _____ 202_ р.

Керівник: канд. техн. наук, асис. кафедри ЕОМ _____ Олег Журило
(підпис)

Завдання прийняв до виконання _____ Петро Лучанінов
(підпис)

РЕФЕРАТ

Пояснювальна записка до курсового проекту: 74 с., 14 рис., 7 табл., 9 джерел.

Об'єктом розробки в даному проекті виступає процес автоматизації обліку товарних надходжень на складських підприємствах. Основною метою роботи є проектування та програмна реалізація інформаційної системи, здатної замінити застарілі паперові методи ведення записів, забезпечуючи при цьому надійність зберігання даних та оперативний доступ до них.

Під час виконання роботи застосовано методи об'єктно-орієнтованого програмування для побудови гнучкої архітектури застосунку. В якості інструментальних засобів обрано мову C++ та середовище Qt Creator з бібліотекою Qt Widgets для реалізації графічного інтерфейсу. Для організації структурованого зберігання інформації використано документо-орієнтовану систему керування базами даних MongoDB, що забезпечує високу швидкодію при роботі з великими масивами даних.

Результатом роботи став програмний продукт "Warehouse System". У системі реалізовано механізм розмежування прав доступу для ролей "Менеджер" та "Комірник", що захищає дані від несанкціонованого видалення. Функціонал програми включає додавання, редагування, видалення та фільтрацію товарних позицій у реальному часі.

Основні техніко-експлуатаційні характеристики розробки включають кросплатформність, високу стабільність роботи та інтуїтивно зрозумілий інтерфейс, який не потребує тривалого навчання персоналу.

Програмний продукт рекомендується до використання на підприємствах малого та середнього бізнесу (сфера логістики, роздрібна торгівля) для оптимізації робочих процесів. Значущість роботи полягає у підвищенні ефективності праці персоналу складу та мінімізації помилок при введенні даних. Прогнозні припущення щодо розвитку проекту передбачають інтеграцію зі сканерами штрих-кодів та розширення аналітичної звітності.

СКЛАД, ОБЛІК ТОВАРІВ, C++, QT, MONGODB, БАЗА ДАНИХ, АВТОМАТИЗАЦІЯ, ПРОГРАМНИЙ ПРОДУКТ.

ЗМІСТ

Вступ.....	6
1 Специфікація проєкту.....	7
1.1. Постановка завдання.....	7
1.2. Вимоги до програмного забезпечення	8
1.3. Математичний опис задачі.....	9
2 Програмна документація	12
2.1 Архітектура програмної системи	12
2.2 Тестування програмної системи	14
2.2.1 Тест-вимоги	14
2.2.2 Тест-план.....	15
2.3 Розгортання програмного продукту.....	22
2.3.1 Вимоги до апаратних засобів.....	22
2.3.2 Вимоги до програмних засобів.....	23
2.3.3 Процедура встановлення компонентів системи.....	23
2.3.4 Можливі проблеми при розгортанні та їх вирішення	25
2.4 Керівництво користувача	25
2.4.1 Призначення програмного продукту.....	26
2.4.2 Використання програмного продукту.....	26
2.4.3 Повідомлення користувачеві	33
Висновки	37
Список використаних джерел	38
Додатки.....	39

ВСТУП

В умовах сьогодення для економічної системи визначальним фактором є стає впровадження інноваційних інформаційних систем. Автоматизація бізнес-процесів дозволяє уникнути рутинної паперової роботи, мінімізувати вплив людського фактору та забезпечити керівництво оперативними даними для прийняття рішень.

Впровадження вузькоспеціалізованих ІТ-рішень стає вирішальним фактором успішного функціонування складського господарства. Точний облік надходження товарів є запорукою фінансової стабільності підприємства, дозволяючи контролювати залишки та уникати пересортиці. Невід'ємною складовою таких систем є надійні засоби збереження даних. Сучасні вимоги до швидкодії диктують необхідність використання спеціалізованих систем керування базами даних, які гарантують цілісність інформації та швидкий доступ до неї.

Метою даного курсового проекту є розробка програмного продукту для автоматизації роботи з базою даних про прихід товарів на склад підприємства. Основні завдання включають аналіз предметної області, проектування зручного графічного інтерфейсу та реалізацію функцій додавання, редагування і пошуку записів.

Результатом роботи став програмний додаток "Warehouse System", призначений для автоматизації робочого місця комірника та менеджера складу. Програма дозволяє вести електронний журнал надходжень та розмежовувати права доступу користувачів. Під час розробки використано мову C++ та бібліотеку Qt Widgets, що забезпечило кросплатформність застосунку. Для збереження даних, задля підвищення надійності та гнучкості порівняно зі звичайними файлами, було обрано документо-орієнтовану СКБД MongoDB.

Виконання даного проекту дозволяє закріпити практичні навички об'єктно-орієнтованого програмування та проектування баз даних, що є необхідним етапом підготовки кваліфікованого фахівця в галузі комп'ютерної інженерії.

1 СПЕЦИФІКАЦІЯ ПРОЄКТУ

1.1. Постановка завдання

Дана курсова робота виконується на підставі завдання на виконання курсового проєкту, затвердженого кафедрою інформаційних систем, згідно з навчальним планом спеціальності «Комп'ютерна інженерія». Тема проєкту відповідає варіанту №1 із переліку рекомендованих тем: «Розроблення програмного продукту для роботи з базою даних про прихід товарів на склад підприємства».

У сучасній логістиці швидкість та точність обробки інформації про рух матеріальних цінностей є критичними факторами успіху. Традиційні методи ведення складського обліку, що базуються на паперових журналах або неструктурованих електронних таблицях, мають низку суттєвих недоліків. До них належать висока ймовірність помилок при ручному введенні даних, складність пошуку необхідної інформації у великих масивах записів, відсутність розмежування прав доступу та ризик втрати даних.

Основним завданням цього проєкту є створення спеціалізованого програмного продукту "Warehouse System", який дозволить автоматизувати процес обліку надходження товарів на склад. Система повинна забезпечувати перехід від застарілих методів до сучасного цифрового обліку з використанням надійної системи керування базами даних (СКБД).

Програмний продукт повинен вирішувати наступні задачі:

- Автоматизація введення даних. Забезпечення зручного інтерфейсу для оперативного внесення інформації про нові партії товарів, що надходять від постачальників.
- Структурування інформації. Зберігання даних у впорядкованому вигляді з чіткою структурою (артикул, назва, кількість, ціна, дата, постачальник).
- Пошук та фільтрація. Реалізація механізмів миттєвого пошуку товарів за ключовими параметрами (наприклад, за назвою або артикулом), що значно скорочує час на інвентаризацію та перевірку залишків.
- Безпека даних. Реалізація механізму розмежування прав доступу з метою унеможливлення стороннього втручання в роботу системи. Система повинна передбачати різні рівні доступу для різних категорій персоналу. У рамках розробки виділено дві основні ролі користувачів системи, потреби яких має задовольняти програмний продукт:
 - Комірник (Worker): Відповідає за фізичний прийом товару та внесення даних у систему. Для цієї ролі критично важливими є простота інтерфейсу та

швидкість виконання операцій додавання записів. Функціонал видалення даних для комірника має бути обмежений задля запобігання випадковим або навмисним втратам інформації.

– Менеджер складу (Manager): Відповідає за загальний контроль та аналітику. Ця роль потребує повного доступу до всіх функцій системи, включаючи редагування, видалення записів та перегляд підсумкової інформації про стан складу.

Технічна реалізація проекту базується на використанні мови програмування C++ та бібліотеки Qt Widgets, що є стандартом для розробки швидких та надійних десктопних застосунків. В якості сховища даних обрано документо-орієнтовану СКБД MongoDB. Такий вибір обумовлений необхідністю забезпечення високої швидкодії при роботі з великими обсягами даних та гнучкістю структури документів, що дозволяє легко масштабувати систему в майбутньому без зміни архітектури програми.

1.2. Вимоги до програмного забезпечення

У цьому підрозділі визначено функціональні та нефункціональні вимоги до програмної системи "Warehouse System", що розробляється.

Функціональні вимоги описують поведінку системи та перетворення вхідних даних, які вона повинна виконувати. Для даного курсового проекту, з урахуванням специфіки використання СКБД MongoDB та розмежування прав доступу, встановлено наступні вимоги:

- Забезпечення з'єднання з сервером бази даних MongoDB та ініціалізація колекцій для зберігання даних.
- Наявність механізму входу в систему за логіном та паролем із визначенням ролі користувача («Менеджер» або «Комірник»).
- Отримання повного списку товарів із бази даних та їх візуалізація у табличному вигляді.
- Можливість створення нових записів про надходження товару із зазначенням усіх характеристик (артикул, назва, ціна, кількість, дата, постачальник).
- Можливість редагування існуючих записів у базі даних.
- Можливість видалення записів із бази даних (функція доступна виключно для ролі «Менеджер»).
- Забезпечення можливості пошуку товарів за ключовими словами у реальному часі.
- Контроль введених користувачем даних перед записом у базу.

- Видача користувачу попереджуючих та інформаційних повідомлень про результати операцій.

Нефункціональні вимоги визначають властивості системи, які не пов'язані прямо з її функціями, але впливають на якість використання.

Вимоги до інтерфейсу користувача:

- Елементи інтерфейсу користувача мають бути виконані англійською мовою.

- Наявність Головного меню та Панелі інструментів, які надають доступ до основних функцій застосунку.

- Дані, отримані з бази даних, повинні відображатися у зручному табличному вигляді.

- Використання модальних діалогових вікон для операцій додавання та редагування даних.

- Наявність стандартних пунктів меню для керування з'єднанням та оновлення даних.

- Наявність пункту "About" ("Про програму"), за допомогою якого користувач має можливість переглянути інформацію про розробника.

Вимоги до архітектури програми:

- Використання об'єктно-орієнтованого підходу та патерну проектування Singleton для класу керування базою даних.

- Використання структур даних із бібліотеки Qt (QVector, QList) та стандартної бібліотеки шаблонів STL для обробки списків об'єктів у пам'яті.

- Використання драйвера mongo-cxx-driver для забезпечення операцій введення-виведення даних між застосунком та СКБД.

- Використання механізму винятків (try-catch) для обробки помилок під час роботи з базою даних та мережею.

Вимоги до вихідного коду застосунку:

- Дотримання єдиного стилю кодування (CamelCase для методів та Snake Case змінних, зрозумілі назви класів).

- Організація коду у вигляді багатофайлового проекту (розділення на заголовні файли .h та файли реалізації .cpp).

- Наявність документаційних коментарів у коді: опис призначення класів, методів, вхідних параметрів та значень, що повертаються.

1.3. Математичний опис задачі

Для формалізації задачі автоматизації складського обліку та розробки відповідного програмного забезпечення доцільно використати апарат теорії

множин та математичної логіки. Такий підхід дає змогу уникнути жорсткої прив'язки до специфіки конкретної СУБД та представити систему як сукупність об'єктів та операцій над ними.

Нехай S — інформаційна система обліку товарів. Стан системи у будь-який дискретний момент часу t визначається скінченною множиною товарів G (Goods), що наявні в базі даних:

$$G = \{g_1, g_2, \dots, g_n\}$$

де n — загальна кількість записів про товари ($n \in \mathbb{N} \cup \{0\}$).

Кожен окремий елемент множини (товар) g_i можна представити у вигляді впорядкованого кортежу атрибутів:

$$g_i = \langle ID_i, Art_i, Name_i, Q_i, Unit_i, P_i, D_i, Sup_i \rangle$$

де прийнято наступні позначення:

- ID_i — унікальний ідентифікатор запису в системі (Primary Key);
- Art_i — артикул товару (текстовий ідентифікатор);
- $Name_i$ — назва товару;
- Q_i — кількість товару. На цю величину накладається обмеження невід'ємності: $Q_i \in \mathbb{R}, Q_i \geq 0$;
- $Unit_i$ — одиниця виміру (шт., кг, л тощо);
- P_i — ціна за одиницю товару. Обмеження: $P_i \in \mathbb{R}, P_i \geq 0$;
- D_i — дата надходження товару;
- Sup_i — інформація про постачальника.

Функціонування системи описується як зміна стану множини G під впливом зовнішніх керуючих дій користувача.

Операція додавання товару (*Add*) призводить до розширення множини G новим елементом g_{new} :

$$G_{t+1} = G_t \cup \{g_{new}\}$$

Операція видалення товару (*Delete*) за ідентифікатором ID_k описується як різниця множин:

$$G_{t+1} = G_t \setminus \{g_k | g_k \in G_t \wedge ID_k = target_{ID}\}$$

Операція пошуку та фільтрації (*Search*) полягає у формуванні підмножини $R \subseteq G$ (*Result set*), елементи якої задовольняють заданому предикату (умові) F :

$$R = \{g_i \in G \mid F(g_i) = true\}$$

У контексті даної курсової роботи предикат F визначається як входження пошукового рядка S_{query} у назву або артикул товару:

$$F(g_i) = (S_{query} \subseteq Name_i) \vee (S_{query} \subseteq Art_i)$$

Однією з функціональних вимог для ролі «Менеджер» є відображення підсумкової інформації. Визначимо математичні залежності вихідних розрахункових даних від вхідних параметрів.

Загальна кількість товарних одиниць на складі (Q_{total}) розраховується як сума кількостей усіх елементів множини G :

$$Q_{total} = \sum_{i=1}^n Q_i$$

Загальна вартість усіх товарів на складі (V_{total}) є сумою добутків кількості кожного товару на його ціну:

$$V_{total} = \sum_{i=1}^n (Q_i \cdot P_i)$$

де:

- Q_i — кількість i -го товару;
- P_i — ціна i -го товару;
- n — поточна потужність множини G (кількість записів).

Розроблена математична модель задовольняє вимогам реалістичності, оскільки вона враховує всі суттєві параметри предметної області (ціна, кількість, унікальність записів) та накладає необхідні обмеження (невід'ємність цін та кількості).

Вимога реалізованості забезпечується тим, що всі наведені операції (об'єднання множин, сумування рядів, логічні порівняння) можуть бути ефективно реалізовані засобами обраної мови програмування C++ та функціоналом СКБД MongoDB з прийнятними часовими витратами.

2 ПРОГРАМНА ДОКУМЕНТАЦІЯ

2.1 Архітектура програмної системи

Програмна система "Warehouse System" спроектована відповідно до принципів об'єктно-орієнтованого програмування (ООП). Архітектура застосунку базується на модульній структурі, де кожен компонент відповідає за окремий аспект функціонування системи: взаємодію з користувачем, обробку бізнес-логіки, збереження даних та візуалізацію аналітики.

Основним архітектурним патерном, використаним при проектуванні графічного інтерфейсу, є модифікація патерну Model-View (Модель-Вигляд), що є стандартом для бібліотеки Qt. Завдяки цьому досягається чітке розмежування рівня візуалізації інформації (зокрема, табличних даних, діаграм та екранних форм) від самої структури даних та механізмів їх обробки.

Для візуалізації статичної структури розробленого програмного забезпечення, відображення ієрархії класів та їх взаємозв'язків побудовано UML-діаграму класів (рис. 2.1). Вона демонструє основні сутності предметної області, їх атрибути, методи та типи відносин (асоціація, залежність).

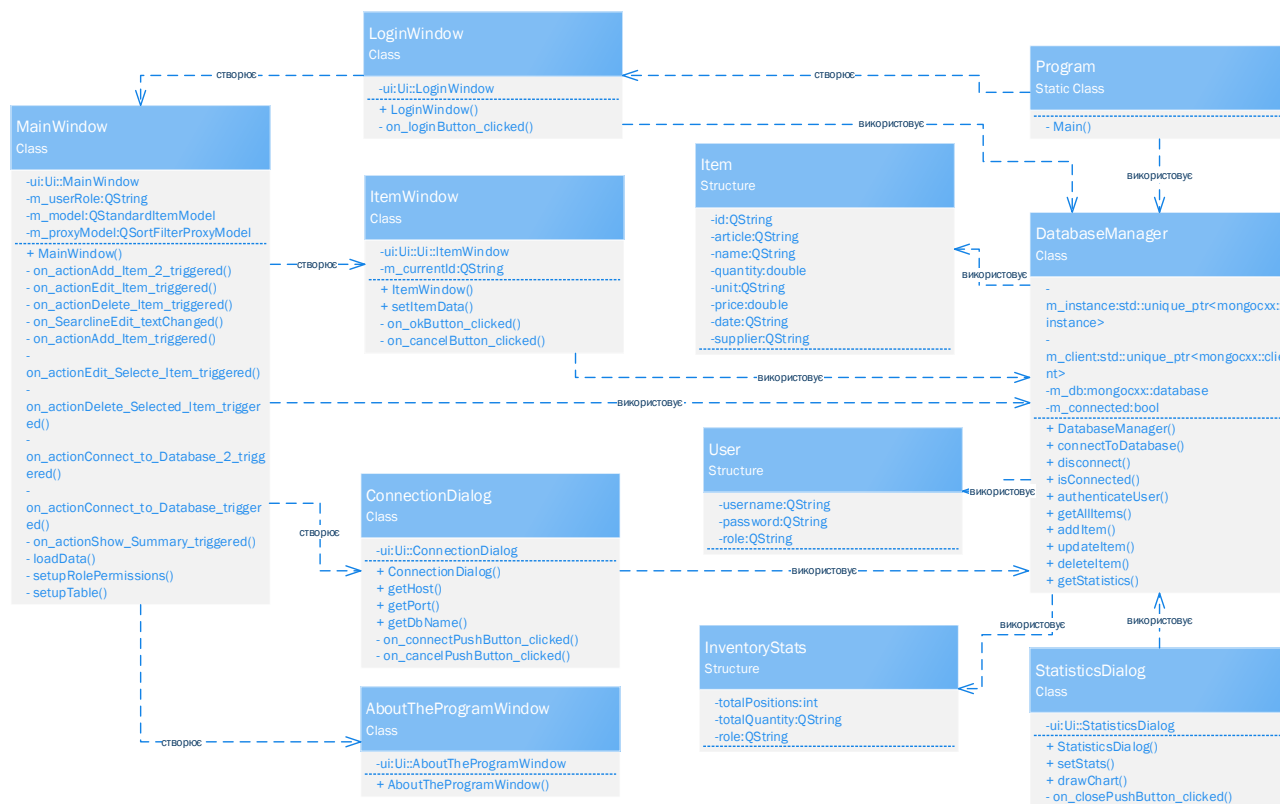


Рисунок 2.1 — UML-діаграма класів програмної системи "Warehouse System"

Опис основних класів та структур, що входять до складу програмної системи:

– `DatabaseManager` – центральний клас бізнес-логіки застосунку, реалізований за патерном проектування «Одинак» (Singleton). Він відповідає за ініціалізацію з'єднання з СКБД MongoDB, керування драйвером `mongosxx` та виконання всіх CRUD-операцій (створення, читання, оновлення, видалення). Клас інкапсулює методи для автентифікації користувачів (`authenticateUser`), отримання списку товарів (`getAllItems`) та розрахунку статистики (`getStatistics`), приховуючи низькорівневу реалізацію запитів від інтерфейсу.

– `Program (Main)` – точка входу в програму. Цей модуль відповідає за ініціалізацію об'єкта `QApplication`, встановлення первинного з'єднання з базою даних через `DatabaseManager` та запуск вікна авторизації `LoginWindow`.

– `LoginWindow` – клас, що відповідає за інтерфейс автентифікації. Він забезпечує взаємодію з користувачем під час входу в систему, перевіряє введені облікові дані та, у разі успіху, ініціює створення головного вікна програми.

– `MainWindow` – основний клас графічного інтерфейсу користувача. Він реалізує логіку відображення табличних даних про товари, використовуючи модель `QStandardItemModel` та проксі-модель `QSortFilterProxyModel` для забезпечення функцій сортування і "живого" пошуку. Клас також керує правами доступу, приховуючи певні елементи керування залежно від ролі поточного користувача (змінна `m_userRole`).

– `ItemWindow` – діалогове вікно, призначене для додавання нових та редагування існуючих товарних позицій. Клас містить логіку валідації вхідних даних (перевірка на порожні поля, коректність числових значень) та взаємодіє з `DatabaseManager` для збереження змін.

– `StatisticsDialog` – клас, що відповідає за формування та візуалізацію аналітичної звітності. Він отримує розраховані дані про стан складу і відображає їх у вигляді числових показників та графіків динаміки надходжень, використовуючи модуль `QtCharts`.

– `ConnectionDialog` – допоміжне діалогове вікно для налаштування параметрів підключення до сервера бази даних (хост, порт, назва бази). Дозволяє динамічно змінювати конфігурацію без перекомпіляції програми.

– `AboutTheProgramWindow` – інформаційне вікно, що містить відомості про розробника, версію програмного продукту та короткий опис призначення системи.

Окрім класів логіки та інтерфейсу, система використовує допоміжні структури даних для типізації інформації:

– `Item` – структура, що описує модель товару. Містить поля для зберігання ідентифікатора, артикула, назви, кількості, ціни, дати надходження та

постачальника. Використовується для передачі даних між шаром бази даних та інтерфейсом.

- User – структура для зберігання даних про користувача системи, включаючи логін, пароль та роль доступу («Менеджер» або «Комірник»).

- InventoryStats – структура, призначена для агрегації підсумкових статистичних даних (загальна кількість позицій, сумарна вартість товарів на складі), які використовуються у модулі звітності.

Повний лістинг вихідного коду програми, що відповідає наведеним класам та реалізує основну бізнес-логіку системи, наведено у Додатку А.

2.2 Тестування програмної системи

Тестування ПЗ — це процедура перевірки працездатності коду, головним завданням якої є пошук невідповідностей та пересвідчення у тому, що продукт задовольняє визначені вимоги. У якості основної стратегії контролю якості розробленої системи "Warehouse System" було визначено методику "чорної скриньки (Black Box Testing). Цей метод передбачає перевірку функціональності без заглиблення у внутрішню структуру коду, базуючись лише на вхідних даних та очікуваних результатах.

Головною метою тестування у даному проєкті є мінімізація кількості дефектів у кінцевому продукті та підтвердження стабільності роботи системи при взаємодії з СКБД MongoDB.

2.2.1 Тест-вимоги

На основі функціональних вимог, викладених у підрозділі 1.2, сформовано наступний перелік тест-вимог (ТВ), які покривають увесь основний функціонал системи:

- Система повинна надавати можливість налаштування параметрів підключення (хост, порт, назва бази) та коректно встановлювати з'єднання з сервером MongoDB. У разі відсутності з'єднання має виводитися відповідне повідомлення.

- Вхід до системи має бути можливим лише за наявності облікового запису в базі даних. Система повинна блокувати доступ при введенні невірних даних та коректно ідентифікувати роль користувача.

- Головне вікно повинно відображати актуальний список товарів у табличному вигляді одразу після входу. Дані мають відповідати записам у базі.

– Система повинна дозволяти створювати нові записи. При цьому має працювати механізм валідації: заборона створення записів з порожніми обов'язковими полями (назва, артикул) або від'ємними числовими значеннями.

– Користувач повинен мати можливість змінювати параметри існуючих товарів. Зміни мають зберігатися в базі даних та миттєво відображатися в таблиці.

– Функція видалення має бути доступна лише користувачам з роллю «Менеджер». Перед видаленням система повинна запитувати підтвердження.

– Система повинна забезпечувати фільтрацію записів у таблиці в реальному часі за входженням підрядка в поля «Назва» або «Артикул».

– Користувачі з роллю «Менеджер» повинні мати доступ до вікна зі статистикою та графіком надходжень. Для інших ролей доступ має бути заблоковано.

2.2.2 Тест-план

Тестовий приклад: № 1.

Призначення: Перевірка механізму налаштування з'єднання та автентифікації користувача.

Тест-вимоги, що перевіряються: № 1, № 2.

Передумови для тесту: Програма запущена, відкрито вікно входу LoginWindow. Сервер бази даних запущено.

Критерій проходження тесту: Успішне встановлення з'єднання та вхід у головне вікно програми.

Таблиця 2.1 – Тестовий приклад № 1

№ з/п	Дія / Етап тестування	Очікувана поведінка системи	Фактичний результат	Статус проходження чергової дії алгоритму (Так/Ні)
1	2	3	4	5
1.	Натиснути кнопку "Login" без введення даних.	Повинно з'явитися повідомлення "Please fill in all fields!" у нижній частині вікна червоним кольором.	Повідомлення відобразилося коректно.	Так

Продовження таблиці 2.1

1	2	3	4	5
2.	Ввести довільний неіснуючий логін та пароль, натиснути "Login".	Повинно з'явитися повідомлення "Incorrect login or password!" або повідомлення про відсутність з'єднання.	Повідомлення про помилку відобразилося.	Так
3.	У меню вибрати пункт "Connection" -> "Connect to Database".	Має відкритися діалогове вікно налаштування підключення "Connection Window".	Діалогове вікно відкрилося.	Так
4.	Ввести параметри: Host: localhost, Port: 27017, DB: Warehouse_System та натиснути "Connect".	Повинно з'явитися спливаюче вікно з повідомленням "Successfully connected to the database!"	З'єднання встановлено успішно, повідомлення отримано.	Так
5.	Ввести коректні дані менеджера (User: manager, Pass: manager) та натиснути "Login".	Вікно входу має закритися, повинно відкритися головне вікно "Warehouse System".	Вхід виконано успішно, головне вікно відкрито.	Так

Відмітка про проходження тесту (пройдено/не пройдено): пройдено.

Тестовий приклад: № 2.

Призначення: Перевірка функції додавання нового товару та валідації вхідних даних.

Тест-вимоги, що перевіряються: № 3, № 4.

Передумови для тесту: Користувач авторизований, відкрито головне вікно.

Критерій проходження тесту: Новий запис коректно зберігається в БД і відображається в таблиці.

Таблиця 2.2 – Тестовий приклад № 2

№ з/п	Дія / Етап тестування	Очікувана поведінка системи	Фактичний результат	Статус проходження чергової дії алгоритму (Так/Ні)
1	2	3	4	5
1.	На панелі інструментів натиснути кнопку "Add Item".	Має відкритися діалогове вікно "Adding a new item" з порожніми полями.	Вікно додавання відкрилося.	Так
2.	Заповнити поле "Quantity" значенням 10, "Price" — 100. Поля "Article" та "Name" залишити порожніми. Натиснути "Ok".	Повинно з'явитися попередження: "The fields 'Article' та 'Name' are required!". Вікно не повинно закриватися.	Попередження отримано, запис не створено.	Так
3.	Заповнити всі поля: Article – TEST-001, Name – Video card, Quantity – 5, Price – 15000. Натиснути "OK".	Діалогове вікно має закритися. У таблиці головного вікна має з'явитися новий рядок із введеними даними.	Вікно закрилося, товар з'явився у кінці таблиці.	Так
4.	Перезапустити програму та увійти знову.	Доданий запис має бути присутнім у таблиці (перевірка збереження в БД).	Запис TEST-001 присутній у таблиці після перезапуску.	Так

Відмітка про проходження тесту (пройдено/не пройдено): пройдено.

Тестовий приклад: № 3.

Призначення: Перевірка функції редагування існуючого товару.

Тест-вимоги, що перевіряються: № 5.

Передумови для тесту: У списку є товар з артикулом TEST-001.

Критерій проходження тесту: Змінені дані коректно оновлюються в системі.

Таблиця 2.3 – Тестовий приклад № 3

№ з/п	Дія / Етап тестування	Очікувана поведінка системи	Фактичний результат	Статус проходження чергової дії алгоритму (Так/Ні)
1	2	3	4	5
1.	Виділити у таблиці рядок з товаром TEST-001. Натиснути кнопку "Edit Item".	Має відкритися вікно "Item Editing", поля якого заповнені поточними даними товару.	Вікно відкрилося, дані завантажено коректно.	Так
2.	Змінити значення поля "Quantity" з 5 на 50, а price на 14500. Натиснути "Update".	Вікно має закритися. У таблиці значення кількості та ціни для цього товару мають змінитися.	Дані в таблиці оновилися успішно.	Так
3.	Перевірити базу даних MongoDB через зовнішній інструмент (MongoDB Compass).	У колекції Data відповідний документ повинен мати поле quantity: 50.	Дані в базі відповідають зміненим у програмі.	Так

Відмітка про проходження тесту (пройдено/не пройдено): пройдений.

Тестовий приклад: № 4.

Призначення: Перевірка розмежування прав доступу для ролі «Комірник».

Тест-вимоги, що перевіряються: № 5, № 8.

Передумови для тесту: Виконано вихід із системи та повторний вхід під логіном worker (пароль worker).

Критерій проходження тесту: Недоступність функцій видалення та перегляду звітів.

Таблиця 2.4 – Тестовий приклад № 4

№ з/п	Дія / Етап тестування	Очікувана поведінка системи	Фактичний результат	Статус проходження чергової дії алгоритму (Так/Ні)
1	2	3	4	5
1.	Перевірити рядок стану (StatusBar) внизу вікна.	Має відображатися напис: "User: Worker".	Роль відображається вірно.	Так
2.	Спробувати знайти на панелі інструментів кнопку "Delete Item".	Кнопка має неактивна (сіра).	Кнопка "Delete" не активна.	Так
3.	У головному меню спробувати натиснути "Reports" -> "Show Summary".	Має з'явитися повідомлення: "Only managers have access to financial statistics!".	Доступ заблоковано, повідомлення отримано.	Так
4.	Спробувати відредагувати товар (натиснути "Edit item").	Вікно редагування має відкритися (комірник має право змінювати кількість).	Вікно редагування доступне.	Так

Відмітка про проходження тесту (пройдено/не пройдено): пройдено.

Тестовий приклад: № 5.

Призначення: Перевірка функції видалення товару (для Менеджера).

Тест-вимоги, що перевіряються: № 6.

Передумови для тесту: Вхід виконано під логіном manager. У списку є тестовий товар.

Критерій проходження тесту: Товар видаляється тільки після підтвердження.

Таблиця 2.5 – Тестовий приклад № 5

№ з/п	Дія / Етап тестування	Очікувана поведінка системи	Фактичний результат	Статус проходження чергової дії алгоритму (Так/Ні)
1	2	3	4	5
1.	Виділити тестовий товар у таблиці. Натиснути кнопку "Delete Item".	Має з'явитися діалогове вікно з питанням: " Are you sure you want to delete this item? This action cannot be undone.".	Вікно підтвердження з'явилося.	Так
2.	У вікні підтвердження натиснути кнопку "No" (Ні).	Вікно закривається, товар залишається у таблиці.	Видалення скасовано, товар на місці.	Так
3.	Знову натиснути "Delete Item", потім натиснути "Yes" (Так).	Вікно закривається. Товар зникає з таблиці. З'являється повідомлення "Товар успішно видалено".	Товар видалено зі списку.	Так

Відмітка про проходження тесту (пройдено/не пройдено): пройдено.

Тестовий приклад: № 6.

Призначення: Перевірка функції живого пошуку та модуля статистики.

Тест-вимоги, що перевіряються: № 7, № 8.

Передумови для тесту: У базі є товари: "Monitor", "Mouse", "Keyboard".
Вхід під manager.

Критерій проходження тесту: Коректна фільтрація списку та відображення графіка.

Таблиця 2.6 – Тестовий приклад № 6

№ з/п	Дія / Етап тестування	Очікувана поведінка системи	Фактичний результат	Статус проходження чергової дії алгоритму (Так/Ні)
1	2	3	4	5
1.	У поле пошуку ввести літеру "М".	У таблиці мають залишитися тільки "Monitor" та "Mouse". "Keyboard" має зникнути.	Фільтрація спрацювала миттєво.	Так
2.	Дописати "о" (вийде "Мо").	У таблиці має залишитися тільки "Monitor".	Залишився один запис.	Так
3.	Очистити поле пошуку.	Мають відобразитися всі товари.	Список відновився повністю.	Так
4.	Відкрити меню "Reports" -> "Show Summary".	Має відкритися вікно "Statistics Window". У ньому відображаються загальні суми та графік (Chart).	Вікно відкрилося, графік побудовано.	Так

Відмітка про проходження тесту (пройдено/не пройдено): пройдено.

Тестових прикладів виконано: 6.

Тестових прикладів пройдено: 6.

У ході проведення комплексного тестування програмної системи "Warehouse System" було перевірено всі ключові аспекти її функціонування. Результати виконання шести тестових сценаріїв підтвердили відповідність розробленого продукту встановленим функціональним вимогам.

Зокрема, було підтверджено стабільність роботи драйвера бази даних MongoDB, коректність виконання операцій додавання, редагування та видалення даних. Особливу увагу було приділено перевірці системи безпеки: тести показали, що розмежування прав доступу між ролями «Менеджер» та «Комірник» працює надійно, блокуючи несанкціонований доступ до критичних функцій.

Графічний інтерфейс продемонстрував високу чутливість (responsiveness), особливо при використанні функції пошуку. Виявлені на ранніх етапах налагодження дрібні неточності у валідації даних були успішно виправлені, що підтверджено успішним проходженням фінальних тестів. Програмний продукт готовий до розгортання та експлуатації.

2.3 Розгортання програмного продукту

Розгортання (Deployment) — це процес встановлення, налаштування та введення в експлуатацію програмного забезпечення на цільових комп'ютерах користувачів. У рамках даного курсового проекту розроблено програмну систему "InStock", яка постачається у вигляді готового до використання дистрибутиву (Portable-версія або Інсталятор).

Оскільки застосунок працює у клієнт-серверній архітектурі (де клієнтом є програма "InStock", а сервером — СКБД MongoDB), процедура розгортання вимагає попередньої підготовки програмного середовища операційної системи. Цей розділ містить вичерпний перелік вимог до апаратного та програмного забезпечення, а також покрокову інструкцію з інсталяції компонентів системи.

2.3.1 Вимоги до апаратних засобів

Програмний продукт не висуває високих вимог до апаратних ресурсів і може функціонувати на більшості сучасних офісних комп'ютерів. Однак для забезпечення стабільної роботи серверної частини бази даних та плавності графічного інтерфейсу рекомендується дотримуватися наведених нижче параметрів.

Мінімальні апаратні вимоги:

1. Центральний процесор (CPU): Архітектура x86-64 (Intel або AMD) з тактовою частотою від 1.6 ГГц. Підтримка набору інструкцій SSE2.
2. Оперативна пам'ять (RAM):
 - Мінімум 4 ГБ загальної пам'яті системи.
 - Рекомендовано 8 ГБ, якщо на цьому ж комп'ютері запускається і клієнтська частина, і сервер бази даних.

3. Дисковий простір (HDD/SSD):

- Для файлів програми: 200 МБ.
- Для сервера СКБД та файлів бази даних: мінімум 1 ГБ вільного простору (обсяг може зростати в процесі накопичення даних). Рекомендується використання SSD-накопичувача для прискорення операцій читання/запису.

4. Відеоадаптер: Інтегрована або дискретна відеокарта з підтримкою OpenGL 2.0 або вище (для коректного відмальовування віджетів Qt).

5. Пристрої введення-виведення: Клавіатура, миша, монітор з роздільною здатністю не менше 1280x720 пікселів.

2.3.2 Вимоги до програмних засобів

Перед початком інсталяції необхідно переконатися, що на комп'ютері встановлено необхідне системне програмне забезпечення.

1. Операційна система: Microsoft Windows 10 (версії 1809 або новішої) або Windows 11. Розрядність системи: 64-bit (x64). Примітка: Робота на 32-бітних системах не підтримується через обмеження сучасних версій MongoDB.

2. Системні бібліотеки Microsoft: Пакет Microsoft Visual C++ Redistributable 2015-2022. Цей пакет необхідний для запуску програм, написаних мовою C++, оскільки він містить бібліотеки часу виконання (runtime libraries), що використовуються як самим додатком, так і драйверами бази даних.

3. Мережеве оточення: Налаштований протокол TCP/IP. Якщо клієнт і сервер знаходяться на різних машинах, необхідно відкрити порт 27017 у брандмауері Windows (Firewall).

2.3.3 Процедура встановлення компонентів системи

Процес розгортання складається з двох послідовних етапів: встановлення серверної частини (СКБД) та розгортання клієнтського застосунку.

Етап 1. Встановлення та налаштування СКБД MongoDB

Оскільки "Warehouse System" не має вбудованої бази даних, а використовує зовнішню професійну СКБД, її необхідно встановити першою.

1. Завантаження: Завантажте інсталяційний пакет MongoDB Community Server (версії 6.0 або 7.0) у форматі .msi з офіційного сайту розробника.

2. Запуск інсталятора: Запустіть завантажений файл. У вікні привітання натисніть "Next".

3. Вибір типу встановлення: Оберіть варіант "Complete" (Повна інсталяція).

4. Налаштування служби (Service Configuration): Це критично важливий крок.

- Встановіть прапорець "Install MongoDB as a Service".
- Це дозволить базі даних запускатися автоматично разом із Windows у фоновому режимі, не вимагаючи ручного запуску щоразу.

5. Завершення: Натисніть "Install" і дочекайтеся завершення процесу.

6. Перевірка: Відкрийте "Диспетчер завдань" (Task Manager) -> вкладка "Служби" (Services) і переконайтеся, що служба MongoDB має статус "Running" (Працює).

Етап 2. Розгортання клієнтського застосунку

Клієнтська частина програми "Warehouse System" постачається у вигляді архіву WarehouseSystem_Release.zip або папки з готовою збіркою, яка вже містить виконуваний файл .exe та всі необхідні динамічні бібліотеки (.dll).

1. Копіювання файлів: Скопіюйте папку з програмою (наприклад, WarehouseSystem) на локальний диск користувача.

- Рекомендований шлях: C:\Program Files\WarehouseSystem (вимагає прав адміністратора) або C:\Users\User\Documents\WarehouseSystem.

2. Перевірка цілісності: Відкрийте папку та переконайтеся у наявності таких ключових файлів:

- WarehouseSystem.exe (Головний файл запуску).
- Qt6Core.dll, Qt6Widgets.dll, Qt6Gui.dll (Бібліотеки інтерфейсу).
- mongocxx.dll, bsoncxx.dll (Драйвери бази даних).
- Папки platforms, styles (Плагіни Qt).

3. Створення ярлика: Для зручності запуску натисніть правою кнопкою миші на файл WarehouseSystem.exe -> "Надіслати" -> "Робочий стіл (створити ярлик)".

Етап 3. Перший запуск та конфігурація

Після розгортання файлів необхідно провести первинне налаштування з'єднання.

1. Запустіть програму через створений ярлик.

2. При першому старті програма спробує підключитися до локального сервера за адресою localhost:27017.

- Якщо Етап 1 (встановлення MongoDB) виконано коректно, програма автоматично створить необхідні структури даних (колекції) і відкрис вікно авторизації.

- Якщо з'єднання відсутнє, з'явиться повідомлення про помилку. У такому разі перевірте, чи запущена служба MongoDB.

3. Якщо база даних знаходиться на іншому комп'ютері в мережі:
 - У вікні налаштувань програми ("Settings") вкажіть IP-адресу сервера та порт.
4. Після успішного підключення система готова до експлуатації. Адміністратор повинен здійснити перший вхід під стандартним логіном (наприклад, admin) та створити облікові записи для персоналу.

2.3.4 Можливі проблеми при розгортанні та їх вирішення

У процесі інсталяції можуть виникнути типові проблеми, шляхи вирішення яких наведено нижче:

1. Помилка "Відсутній файл VCRUNTIME140.dll" або "MSVCP140.dll":
 - Причина: На комп'ютері не встановлено пакет Visual C++ Redistributable.
 - Рішення: Завантажити та встановити файл vc_redist.x64.exe з сайту Microsoft.
2. Помилка "Connection Refused" (У з'єднанні відмовлено):
 - Причина: Служба MongoDB зупинена або блокується антивірусом/брандмауером.
 - Рішення: Перезапустити службу MongoDB через консоль "Services.msc" або додати виключення для порту 27017 у брандмауері.
3. Помилка "Application failed to start correctly (0xc000007b)":
 - Причина: Невідповідність розрядності бібліотек (спроба запуску 64-бітної програми з 32-бітними DLL або навпаки).
 - Рішення: Переконаватися, що використовується коректна версія дистрибутиву, що відповідає архітектурі ОС.

Виконання наведених інструкцій гарантує успішне розгортання та стабільну роботу програмного комплексу "InStock" на робочому місці користувача.

2.4 Керівництво користувача

Створене програмне рішення "Warehouse System" є інструментом для автоматизації складського обліку, орієнтованим на користувачів із базовими навичками роботи з персональним комп'ютером. Даний розділ містить детальну інформацію про призначення системи, покрокову інструкцію з експлуатації та опис можливих системних повідомлень.

2.4.1 Призначення програмного продукту

Програмний комплекс "Warehouse System" призначений для ведення оперативного обліку надходження товарно-матеріальних цінностей на складах підприємств малого та середнього бізнесу. Система замінює застарілі методи ведення записів (паперові журнали, неструктуровані таблиці Excel) на сучасний підхід із використанням централізованої бази даних.

Основні функції програми:

- Облік надходжень: Реєстрація нових партій товару із зазначенням артикула, назви, кількості, ціни за одиницю, дати прийому та постачальника.
- Керування даними: Можливість редагування інформації про товари у разі виявлення помилок та видалення застарілих або помилкових записів (тільки для адміністративного персоналу).
- Пошук та навігація: Миттєвий пошук необхідного товару у базі даних за фрагментом назви або артикула, що значно пришвидшує проведення інвентаризації.
- Аналітика: Автоматичний розрахунок загальної вартості складу та візуалізація динаміки надходжень товарів у часі (графіки).
- Безпека: Захист даних від несанкціонованого доступу та розмежування прав користувачів.

Категорії користувачів: Система розрахована на роботу двох типів користувачів:

- Комірник (Worker): Відповідає за фізичний прийом товару. Має доступ до перегляду списку, пошуку та додавання нових позицій. Функції видалення та перегляду фінансової статистики для нього заблоковані.
- Менеджер (Manager): Здійснює загальне керівництво. Має повний доступ до всіх функцій системи, включаючи видалення записів, редагування будь-яких полів та перегляд аналітичних звітів.

Експлуатація програми сприяє суттєвому скороченню часових витрат на опрацювання окремих документів, нівелює ризики виникнення неточностей через помилки персоналу, а також забезпечує керівництво актуальною інформацією про стан матеріальних активів підприємства.

2.4.2 Використання програмного продукту

Експлуатація програмного комплексу "Warehouse System" починається із запуску виконуваного файлу WarehouseSystem.exe з робочого столу або папки встановлення.

2.4.2.1 Запуск та ініціалізація системи

При запуску програма автоматично виконує процедуру ініціалізації:

- Завантажуються необхідні драйвери та бібліотеки Qt.
- Відбувається спроба встановлення з'єднання з локальним сервером бази даних MongoDB (порт 27017 за замовчуванням).
- Якщо з'єднання встановлено успішно, на екрані з'являється вікно авторизації. У разі відсутності зв'язку з базою даних, користувач отримає відповідне системне повідомлення, і подальша робота буде неможливою до відновлення роботи сервера СКБД.

2.4.2.2 Авторизація в системі

Доступ до функціоналу програми захищено системою автентифікації. Вікно входу (рис. 2.2) є першим інтерфейсним елементом, з яким взаємодіє користувач.

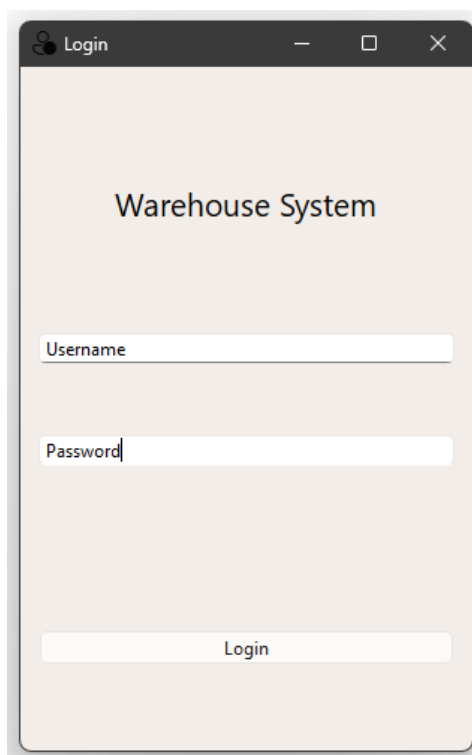


Рисунок 2.2 — Вікно авторизації користувача

Порядок дій:

- У полі "Username" введіть свій особистий логін (наприклад, manager або worker).
- У полі "Password" введіть пароль доступу.
- Натисніть кнопку "Login".

Система звірить введені дані з обліковими записами в базі даних. Якщо пару логін/пароль знайдено, вікно авторизації закриється, і завантажиться головне вікно програми з правами, що відповідають вашій ролі.

2.4.2.3 Робота з головним вікном

Після успішного входу відкривається основний робочий простір (рис. 2.3). Інтерфейс спроектовано таким чином, щоб надати швидкий доступ до всіх ключових функцій.

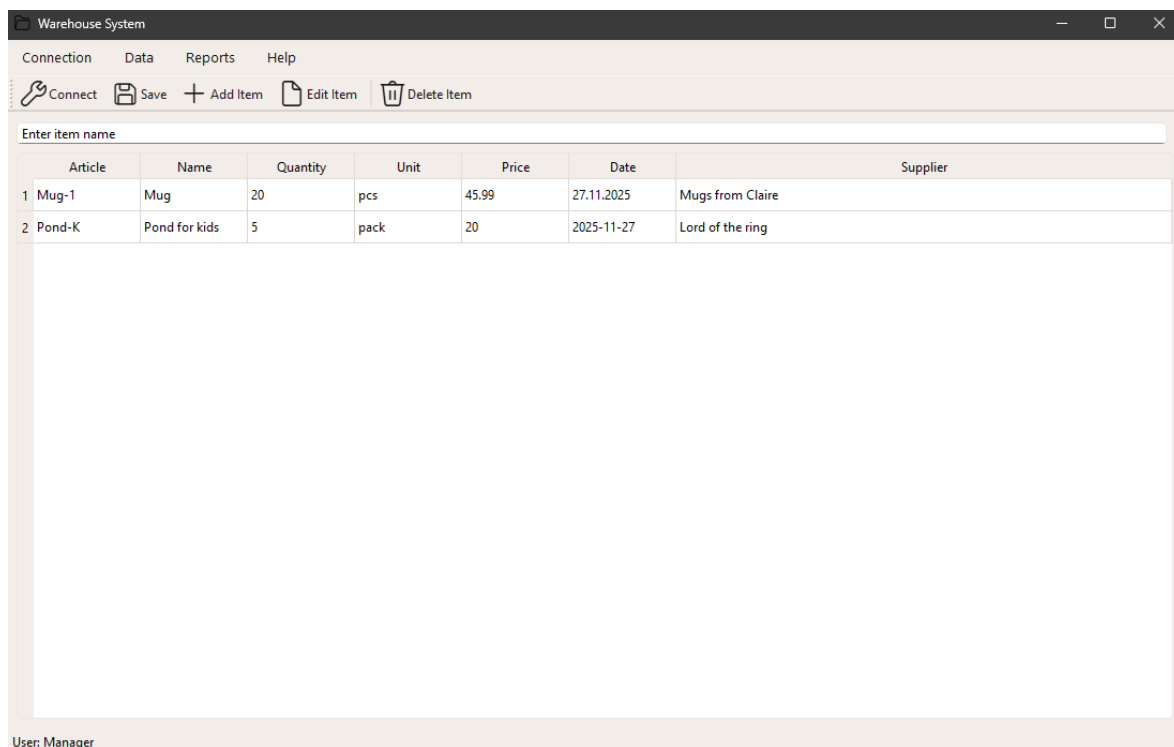


Рисунок 2.3 — Інтерфейс головного вікна програми

Основні елементи керування:

- Панель меню (Menu Bar): Знаходиться у верхній частині вікна і містить пункти:
 - Connection — керування підключенням та вихід з програми.
 - Data — керування елементами в базі даних.
 - Reports — доступ до статистики (тільки для Менеджерів).
 - Help — інформація про програму.
- Панель інструментів (Toolbar): Містить іконки для швидкого виклику операцій підключення (Connect), додавання ("Add item"), редагування ("Edit item") та видалення ("Delete item").

– Таблиця товарів (Data Grid): Центральна частина вікна, де відображається повний перелік товарів. Таблиця автоматично оновлюється при будь-яких змінах у базі.

– Рядок стану (Status Bar): У нижній частині вікна відображається поточна роль користувача (наприклад, "User: Manager") та статус підключення.

2.4.2.4 Пошук та фільтрація даних

Для швидкого знаходження конкретного товару серед великого масиву даних передбачено функцію "живого" пошуку.

Алгоритм пошуку:

- Активуйте поле пошуку, розташоване над таблицею.
- Почніть вводити відомі дані про товар (це може бути частина артикула, наприклад "A-10", або частина назви, наприклад "Mug").
- Програма автоматично фільтрує список у реальному часі. Результати оновлюються після введення кожного символу (рис. 2.4).
- Щоб скинути фільтр і побачити всі товари, просто очистіть поле пошуку.

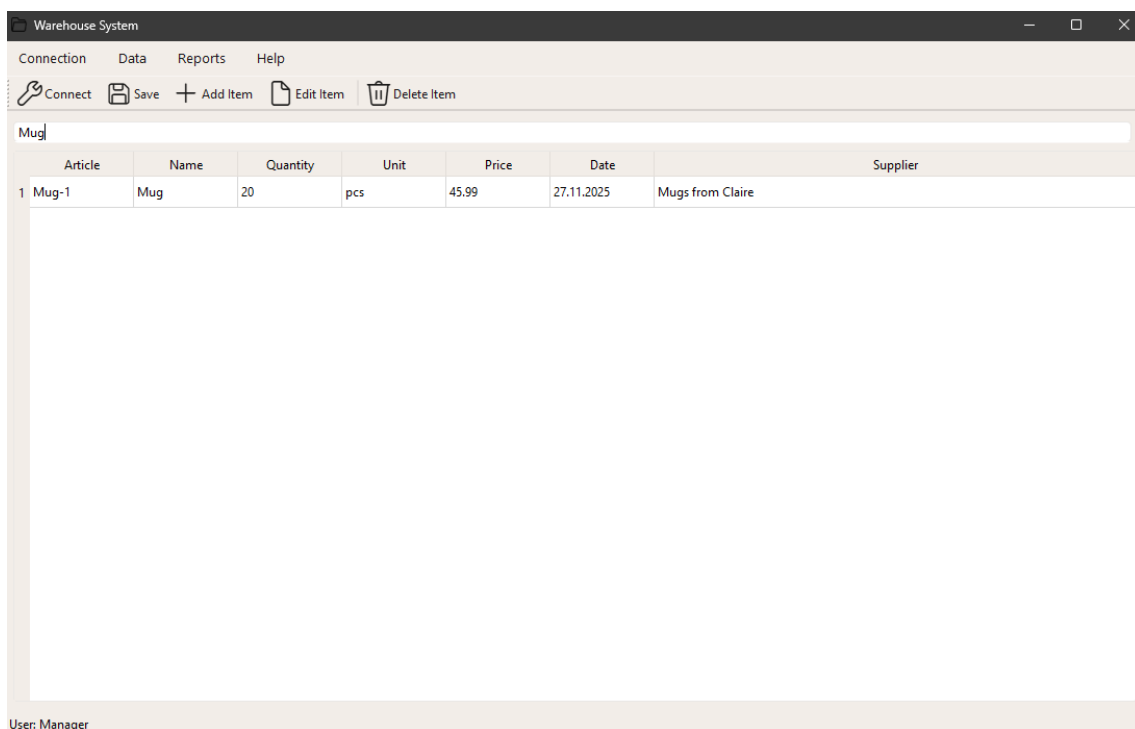


Рисунок 2.4 — Результат фільтрації списку товарів

2.4.2.5 Реєстрація надходження товару

Додавання нової позиції в базу даних виконується через спеціалізовану форму.

Покрокова інструкція:

- Натисніть кнопку "Add Item" на панелі інструментів.
- У діалоговому вікні, що відкрилося (рис. 2.5), заповніть облікову картку товару:
 - Article (Артикул): Введіть унікальний код товару. Це поле є обов'язковим.
 - Name (Назва): Введіть повне найменування товару. Поле обов'язкове.
 - Quantity (Кількість): Вкажіть кількість одиниць. Використовуйте стрілки або введіть число вручну.
 - Unit (Од. вим.): Оберіть одиницю виміру зі списку (шт, кг, л, м).
 - Price (Ціна): Вкажіть ціну за одиницю у гривнях.
 - Supplier (Постачальник): Вкажіть назву компанії-постачальника.
 - Date (Дата): Дата встановлюється автоматично на сьогоднішню, але її можна змінити через календар.
- Перевірте правильність даних та натисніть кнопку "ОК".
- Якщо всі дані коректні, вікно закриється, а новий товар з'явиться у таблиці.

The image shows a software dialog box titled "Adding a new item". It contains several input fields for adding a new item to a database. The fields are: "Article" (a text input), "Name" (a text input), "Quantity" (a numeric input with up and down arrows), "Unit" (a dropdown menu currently showing "pcs"), "Price" (a numeric input with up and down arrows), "Date" (a date picker showing "13.12.2025"), and "Supplier" (a text input). At the bottom of the dialog are two buttons: "Ok" and "Cancel".

Рисунок 2.5 — Форма додавання нового товару

2.4.2.6 Редагування даних

Якщо при введенні було допущено помилку або змінилися характеристики товару (наприклад, ціна), запис можна відредагувати.

- Знайдіть потрібний товар у таблиці та виділіть його кліком миші (рядок підсвітиться синім кольором).
- Натисніть кнопку "Edit Item".
- Відкриється вікно, аналогічне вікну додавання, але поля будуть вже заповнені поточними даними.
- Внесіть необхідні правки.
- Натисніть кнопку "Update" (або "OK"). Зміни миттєво збережуться в базі даних.

2.4.2.7 Видалення товару (Адміністрування)

Примітка: Ця функція доступна тільки для користувачів з правами "Менеджер".

- Виділіть товар, який підлягає списанню або видаленню.
- Натисніть кнопку "Delete Item" (іконка "Кошик").
- Система видасть запит на підтвердження операції (рис. 2.6), щоб уникнути випадкової втрати даних.
- Натисніть "Yes", щоб підтвердити видалення. Запис буде безповоротно вилучено з бази даних.

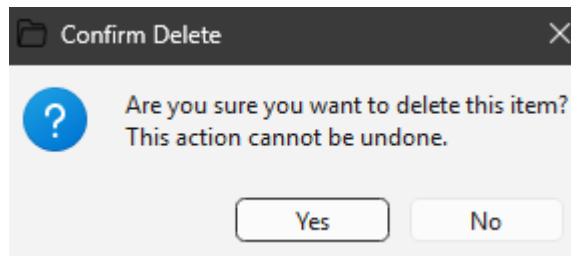


Рисунок 2.6 — Діалог підтвердження видалення

2.4.2.8 Аналітична звітність

Менеджери мають можливість переглядати зведену інформацію про стан складу.

- Перейдіть у меню "Reports" та оберіть пункт "Summary".
- Відкриється вікно аналітики (рис. 2.7), яке містить:
 - Текстовий звіт: Загальна кількість позицій, сумарна кількість одиниць товару та загальна грошова вартість складу.
 - Графічний звіт: Інтерактивний графік, що показує динаміку надходження товарів у часі (вісь X — дати, вісь Y — кількість).
- Ця інформація дозволяє оцінити обертання товарів та поточні фінансові показники.



Рисунок 2.7 — Вікно фінансової статистики та графік

2.4.2.9 Зміна параметрів підключення (Опціонально)

Якщо виникає потреба підключитися до іншого сервера бази даних (наприклад, віддаленого) вже після входу в систему:

- Оберіть в меню пункт "Settings" -> "Connection Settings".
- У вікні налаштувань (рис. 2.8) змініть IP-адресу хоста або порт.
- Натисніть "Connect". Програма перепідключиться до нової бази даних та оновить таблицю товарів.

The figure shows a "Connection Window" dialog box. It contains the following fields and controls:

- A text field with "localhost" and a dropdown menu showing "27017".
- A text field with "Warehouse_System".
- An empty text field.
- Two buttons: "Connect" and "Cancel".

Рисунок 2.8 — Налаштування параметрів з'єднання

2.4.2.10 Завершення роботи

Для виходу з програми:

- Натисніть меню "Connection" -> "Exit".
- Програма коректно закриє з'єднання з базою даних, звільнить пам'ять та завершить роботу.

2.4.3 Повідомлення користувачеві

У процесі роботи програмного продукту можуть виникати ситуації, що вимагають уваги користувача. Система взаємодіє з оператором за допомогою діалогових вікон повідомлень. Нижче наведено опис основних повідомлень, їхні причини та необхідні дії користувача.

2.4.3.1 Повідомлення про помилку підключення

Текст повідомлення: "Connection failed".

Вигляд:

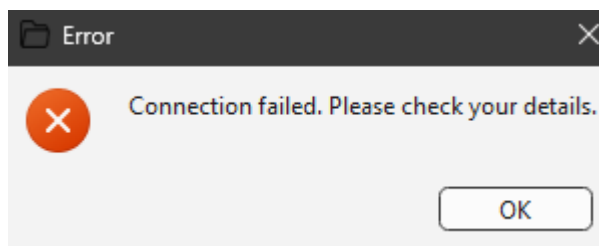


Рисунок 2.9 — Повідомлення про помилку з'єднання

Причина: Програма не може встановити зв'язок із сервером MongoDB. Можливі причини: сервер вимкнено, вказано невірний хост або порт, відсутнє мережеве з'єднання.

Дії користувача: Перевірити, чи запущено службу MongoDB. Перевірити налаштування у меню "Connection Settings".

2.4.3.2 Повідомлення про помилку авторизації

Текст повідомлення: " Incorrect login or password!".

Вигляд:



Рисунок 2.10 – Повідомлення про помилку авторизації

Причина: Користувач ввів облікові дані, які відсутні в базі даних.

Дії користувача: Перевірити правильність введення логіна та пароля (розкладка клавіатури, Caps Lock) та спробувати ще раз.

2.4.3.3 Попередження валідації даних

Текст повідомлення: "Поля 'Артикул' та 'Назва' є обов'язковими!".

Вигляд:



Рисунок 2.11 — Попередження про незаповнені поля

Причина: При додаванні або редагуванні товару користувач залишив ключові поля порожніми.

Дії користувача: Заповнити всі обов'язкові поля форми та повторити збереження.

2.4.3.4 Запит на підтвердження видалення

Текст повідомлення: "Are you sure you want to delete this item? This action cannot be undone."

Вигляд:

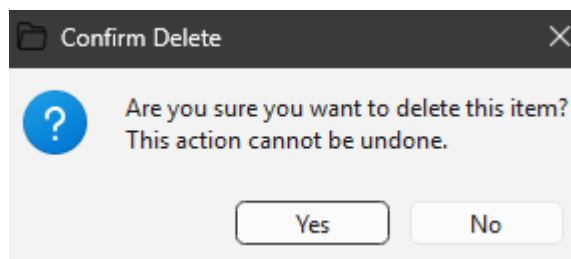


Рисунок 2.12 — Запит на підтвердження видалення

Причина: Спроба видалення товару. Захист від випадкового натискання.

Дії користувача: Натиснути "Yes" для видалення або "No" для скасування операції.

2.4.3.5 Повідомлення про обмеження прав доступу

Текст повідомлення: "Доступ заборонено! Тільки менеджери мають доступ до фінансової статистики!".

Вигляд:

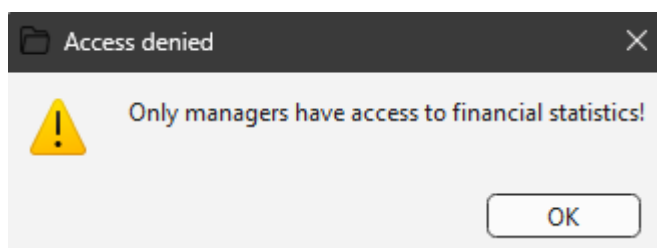


Рисунок 2.13 – Повідомлення про обмеження прав доступу

Причина: Користувач із роллю "Комірник" намагається відкрити звіт або видалити товар.

Дії користувача: Для виконання цієї дії необхідно увійти в систему під обліковим записом Менеджера.

2.4.3.6 Інформаційне повідомлення про успіх

Текст повідомлення: "Item added successfully!".

Вигляд:

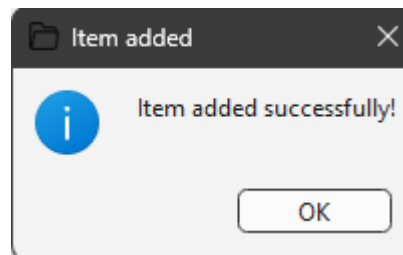


Рисунок 2.14 – Інформаційне повідомлення про успіх

Причина: Підтвердження успішного завершення операції.

Дії користувача: Натиснути "ОК" і продовжити роботу.

ВИСНОВКИ

У ході виконання курсового проекту розроблено програмну систему "Warehouse System", призначену для автоматизації складського обліку на підприємствах малого та середнього бізнесу. Головною метою роботи було створення надійного інструменту, здатного замінити застарілі паперові методи ведення записів та забезпечити оперативний доступ до даних.

Вибір технологічного стека, що включає мову C++, бібліотеку Qt Widgets та СКБД MongoDB, виявився повністю виправданим. Використання документо-орієнтованої бази даних замість традиційних файлів дозволило досягти високої швидкодії при обробці пошукових запитів та гарантувати цілісність інформації. Архітектура застосунку, побудована на патернах Singleton та Model-View, забезпечує гнучкість та легкість у супроводі коду.

З функціональної точки зору система повністю відповідає технічному завданню. Реалізовано повний цикл операцій з даними (створення, читання, оновлення, видалення), механізм "живого" пошуку та рольову модель доступу. Розмежування прав між «Менеджером» та «Комірником» дозволяє ефективно захистити критично важливу інформацію від випадкового видалення.

Серед обмежень розробленого продукту варто відзначити необхідність інсталяції клієнтського застосунку на кожне робоче місце та відсутність вбудованої функції експорту звітів у формат Excel. Усунення цих недоліків та реалізація мережевої взаємодії є пріоритетними напрямками для подальшого розвитку проекту.

Практична цінність роботи полягає у створенні готового до впровадження продукту, використання якого дозволяє скоротити час на обробку накладних та мінімізувати помилки, пов'язані з людським фактором. Економічний ефект досягається за рахунок оптимізації робочого часу персоналу та підвищення точності обліку матеріальних цінностей.

Таким чином, мета курсового проекту досягнута, а створене програмне забезпечення демонструє ефективність поєднання класичних підходів до розробки десктопних застосунків із сучасними технологіями баз даних.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Парфьонов Ю. Е., Федорченко В. М., Щербаков О. В. Об'єктно-орієнтоване програмування : методичні рекомендації до виконання курсового проекту [Електронний ресурс]. – Режим доступу: <https://repository.hneu.edu.ua/bitstream/123456789/21063/1/2018.pdf>.
2. Acode. Уроки по C++ [Електронний ресурс]. – Режим доступу: <https://acode.com.ua/uroki-po-cpp/>.
3. Qt Documentation [Електронний ресурс]. – Режим доступу: <https://doc.qt.io/>.
4. MongoDB Manual [Електронний ресурс]. – Режим доступу: <https://www.mongodb.com/docs/manual/>.

ДОДАТКИ

Додаток А

Лістингу коду файлу main.cpp

```
#include "loginwindow.h"
#include "databasemanager.h" // <--- Наш новий клас
#include <QApplication>
#include <QMessageBox>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    //Підключаємо базу даних
    if (!DatabaseManager::instance().connectToDatabase("localhost", 27017,
"Warehouse_System", "", "")) {
        QMessageBox::critical(nullptr, "Error", "Failed to connect to database!");
        return -1;
    }

    // Запускаємо логін
    LoginWindow w;
    w.show();

    return a.exec();
}
```

Лістингу коду файлу loginwindow.h

```
#ifndef LOGINWINDOW_H
#define LOGINWINDOW_H

#include <QWidget>

namespace Ui {
class LoginWindow;}
#endif
```

```
class LoginWindow : public QWidget
{
    Q_OBJECT

public:
    explicit LoginWindow(QWidget *parent = nullptr);
    ~LoginWindow();

private slots:
    void on_loginButton_clicked();

private:
    Ui::LoginWindow *ui;
};

#endif // LOGINWINDOW_H
```

Лістингу коду файлу loginwindow.cpp

```
#include "loginwindow.h"
#include "databasemanager.h"
#include "mainwindow.h"
#include "ui_loginwindow.h"

LoginWindow::LoginWindow(QWidget *parent)
    : QWidget(parent)
    , ui(new Ui::LoginWindow)
{
    ui->setupUi(this);
}

LoginWindow::~~LoginWindow()
{
    delete ui;
}
```



```
//Натискання кнопки login
void LoginWindow::on_loginButton_clicked()
{
    // Очищаємо попередні повідомлення про помилки
    ui->warningLabel->clear();

    // Отримуємо текст з полів (використовуємо твої назви!)
    QString username = ui->usernameEdit->text().trimmed();
    QString password = ui->passwordEdit->text();

    // Перевірка на порожнечу
    if (username.isEmpty() || password.isEmpty()) {
        ui->warningLabel->setText("Please fill in all fields!");
        ui->warningLabel->setStyleSheet("color: red; font-weight: bold;");
        return;
    }

    // Звертаємося до бази даних
    auto role = DatabaseManager::instance().authenticateUser(username, password);

    // Перевіряємо результат
    if (role.has_value()) {
        // --- УСПІХ ---
        QString userRole = role.value(); // "Manager" або "Worker"

        MainWindow *mainWindow = new MainWindow(userRole);
        mainWindow->show();

        // Закриваємо вікно логіну
        this->close();
    } else {
        // --- НЕВДАЧА ---
        ui->warningLabel->setText("Incorrect login or password!");
        ui->warningLabel->setStyleSheet("color: red; font-weight: bold;");
    }
}
```

```

    // Очистка паролю для зручності
    ui->passwordEdit->clear();
    ui->passwordEdit->setFocus();
}
}

```

Лістингу коду файлу mainwindow.h

```

#ifndef MAINWINDOW_H
#define MAINWINDOW_H
#include <QStandardItemModel>
#include <QMainWindow>
#include <QSortFilterProxyModel>

QT_BEGIN_NAMESPACE
namespace Ui {
class MainWindow;
}
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(const QString &role, QWidget *parent = nullptr);
    ~MainWindow();

private slots:
    void on_actionAdd_Item_2_triggered();

    void on_actionEdit_Item_triggered();

    void on_actionDelete_Item_triggered();

    void on_SearchlineEdit_textChanged(const QString &arg1);

```

```
void on_actionAdd_Item_triggered();

void on_actionEdit_Selecte_Item_triggered();

void on_actionDelete_Selected_Item_triggered();

void on_actionConnect_to_Database_2_triggered();

void on_actionConnect_to_Database_triggered();

void on_actionShow_Summary_triggered();

private:
    Ui::MainWindow *ui;
    QString m_userRole;
    QStandardItemModel *m_model;
    QSortFilterProxyModel *m_proxyModel;
    void loadData();
    void setupRolePermissions();
    void setupTable();

};
#endif // MAINWINDOW_H
```

Лістингу коду файлу mainwindow.cpp

```
#include "mainwindow.h"
#include "../ui_mainwindow.h"
#include "databasemanager.h"
#include "itemwindow.h"
#include "connectiondialog.h"
#include "statisticsdialog.h"
#include <QMessageBox>

MainWindow::MainWindow(const QString &role, QWidget *parent)
```

```
: QMainWindow(parent)
, ui(new Ui::MainWindow)
, m_userRole(role)
{
    ui->setupUi(this);
    m_model = new QStandardItemModel(this);
    m_proxyModel = new QSortFilterProxyModel(this);
    m_proxyModel->setSourceModel(m_model);
    m_proxyModel->setFilterKeyColumn(-1);
    m_proxyModel->setFilterCaseSensitivity(Qt::CaseInsensitive);
    ui->tableView->setModel(m_proxyModel);
    setupTable();
    setupRolePermissions();
    loadData();
}
```

```
MainWindow::~MainWindow()
{
    delete ui;
}
```

// Встановлюємо інтерфейс користувача за ролью

```
void MainWindow::setupRolePermissions()
```

```
{
    // Показуємо роль у статус-барі (внизу вікна)
    ui->statusbar->showMessage("User: " + m_userRole);

    // Якщо це звичайний робітник - ховаємо небезпечні кнопки
    if (m_userRole == "Worker") {
        ui->actionDelete_Selected_Item->setEnabled(false);
        ui->actionDelete_Item->setEnabled(false);
    }
}
```

//Налаштовуємо таблицю

```
void MainWindow::setupTable()
{
    // Задаємо заголовки стовпців
    m_model->setHorizontalHeaderLabels({
        "ID", "Article", "Name", "Quantity", "Unit", "Price", "Date", "Supplier"
    });

    // Ховаємо колонку ID (вона потрібна програмі, але не користувачу)
    // (Припустимо, ID - це 0-й стовпець)
    ui->tableView->setColumnHidden(0, true);

    // Розтягуємо останню колонку
    ui->tableView->horizontalHeader()->setStretchLastSection(true);

    // Забороняємо редагувати комірки прямо в таблиці (тільки через кнопку Edit)
    ui->tableView->setEditTriggers(QAbstractItemView::NoEditTriggers);

    // Виділяти одразу весь рядок, а не одну клітинку
    ui->tableView->setSelectionBehavior(QAbstractItemView::SelectRows);
    ui->tableView->setSelectionMode(QAbstractItemView::SingleSelection);
}

//Заповнюємо таблицю даними
void MainWindow::loadData()
{
    // Очищаємо старі дані (але залишаємо заголовки)
    m_model->removeRows(0, m_model->rowCount());

    // Отримуємо дані з бази
    QVector<Item> items = DatabaseManager::instance().getAllItems();

    // Заповнюємо модель
    for (const Item& item : items) {
        QList<QStandardItem*> row;
```

```
// Створюємо комірки
row << new QStandardItem(item.id);
row << new QStandardItem(item.article);
row << new QStandardItem(item.name);

QStandardItem* qtyItem = new QStandardItem();
qtyItem->setData(item.quantity, Qt::DisplayRole);
row << qtyItem;

row << new QStandardItem(item.unit);

QStandardItem* priceItem = new QStandardItem();
priceItem->setData(item.price, Qt::DisplayRole);
row << priceItem;

row << new QStandardItem(item.date);
row << new QStandardItem(item.supplier);

// Додаємо рядок у модель
m_model->appendRow(row);
}
}

void MainWindow::on_actionAdd_Item_2_triggered()
{
    // Створюємо вікно
    ItemWindow window(this);

    // Встановлюємо заголовок вікна
    window.setWindowTitle("Adding a new item");

    // Відкриваємо вікно і чекаємо
    if (window.exec() == QDialog::Accepted) {
        // Повідомлення про успішне виконання
        QMessageBox::information(this, "Item added",
```

```
        "Item added successfully!");
    // Якщо користувач успішно додав товар і натиснув ОК,
    // перезавантажуємо таблицю, щоб побачити новий запис.
    loadData();
}
}

void MainWindow::on_actionEdit_Item_triggered()
{
    auto selectionModel = ui->tableView->selectionModel();
    auto selectedRows = selectionModel->selectedRows();
    //Повідомлення про пусте поле
    if (selectedRows.isEmpty()) {
        QMessageBox::warning(this, "Warning", "Please select a product to edit!");
        return;
    }

    // Отримуємо номер рядка (беремо перший виділений, якщо їх декілька)
    // Отримуємо індекс з ПРОКСІ (те, що бачить юзер)
    QModelIndex proxyIndex = selectedRows.first();

    // --- МАГІЯ: Перетворюємо його на реальний індекс моделі ---
    QModelIndex sourceIndex = m_proxyModel->mapToSource(proxyIndex);
    int row = sourceIndex.row();

    // Збираємо дані з моделі назад у структуру Item
    Item item;

    item.id      = m_model->item(row, 0)->text();
    item.article = m_model->item(row, 1)->text();
    item.name    = m_model->item(row, 2)->text();
    item.quantity = m_model->item(row, 3)->data(Qt::DisplayRole).toDouble();
    item.unit    = m_model->item(row, 4)->text();
    item.price   = m_model->item(row, 5)->data(Qt::DisplayRole).toDouble();
}
```

```
item.date    = m_model->item(row, 6)->text();
item.supplier = m_model->item(row, 7)->text();

// Відкриваємо вікно редагування
ItemWindow window(this);

// Передаємо зібраний item у вікно (щоб заповнити поля)
window.setItemData(item);

// Запускаємо вікно і чекаємо результату
if (window.exec() == QDialog::Accepted) {
    //Повідомлення про успішне виконання
    QMessageBox::information(this, "Item added",
                             "Item edited successfully!");
    // Якщо користувач натиснув "Оновити", перезавантажуємо таблицю,
    // щоб побачити зміни
    loadData();
}
}

void MainWindow::on_actionDelete_Item_triggered()
{
    // Перевіряємо, чи виділений рядок
    auto selectionModel = ui->tableView->selectionModel();
    auto selectedRows = selectionModel->selectedRows();
    //Повідомлення про пустий рядок
    if (selectedRows.isEmpty()) {
        QMessageBox::warning(this, "Attention", "Please select the product to delete!");
        return;
    }

    // Запитуємо підтвердження
    QMessageBox::StandardButton reply;
    reply = QMessageBox::question(this, "Confirm Delete",
```


"Are you sure you want to delete this item?\nThis action cannot be undone.",

QMessageBox::Yes | QMessageBox::No);

```
if (reply == QMessageBox::No) {  
    return;  
}
```

// Отримуємо ID з прихованої колонки

```
QModelIndex proxyIndex = selectedRows.first();
```

// Перетворення індексу

```
QModelIndex sourceIndex = m_proxyModel->mapToSource(proxyIndex);
```

```
int row = sourceIndex.row();
```

```
QString id = m_model->item(row, 0)->text();
```

// Викликаємо видалення

```
if (DatabaseManager::instance().deleteItem(id)) {
```

 // Якщо успішно — оновлюємо таблицю

```
    loadData();
```

 // Повідомлення про успішне виконання

```
    QMessageBox::information(this, "Success", "Item successfully deleted.");
```

```
} else {
```

 // Повідомлення про неуспішне виконання

```
    QMessageBox::critical(this, "Error", "Failed to delete product from database.");
```

```
}
```

```
}
```

//Функція пошуку таблиці

```
void MainWindow::on_SearchlineEdit_textChanged(const QString &arg1)
```

```
{
```

```
    m_proxyModel->setFilterFixedString(arg1);
```

```
}
```

```
void MainWindow::on_actionAdd_Item_triggered()
{
    // Створюємо вікно
    ItemWindow window(this);

    // Встановлюємо заголовок вікна
    window.setWindowTitle("Adding a new item");

    // Відкриваємо вікно і чекаємо
    if (window.exec() == QDialog::Accepted) {
        // Повідомлення про успішне виконання
        QMessageBox::information(this, "Item added",
                                "Item added successfully!");
        // Якщо користувач успішно додав товар і натиснув ОК,
        // перезавантажуємо таблицю, щоб побачити новий запис.
        loadData();
    }
}

void MainWindow::on_actionEdit_Selecte_Item_triggered()
{
    auto selectionModel = ui->tableView->selectionModel();
    auto selectedRows = selectionModel->selectedRows();
    //Повідомлення про пусте поле
    if (selectedRows.isEmpty()) {
        QMessageBox::warning(this, "Warning", "Please select a product to edit!");
        return;
    }

    // Отримуємо номер рядка (беремо перший виділений, якщо їх декілька)
    // Отримуємо індекс з ПРОКСІ (те, що бачить юзер)
    QModelIndex proxyIndex = selectedRows.first();

    // --- МАГІЯ: Перетворюємо його на реальний індекс моделі ---
```

```
QModelIndex sourceIndex = m_proxyModel->mapToSource(proxyIndex);
int row = sourceIndex.row();

// Збираємо дані з моделі назад у структуру Item
Item item;

item.id      = m_model->item(row, 0)->text();
item.article = m_model->item(row, 1)->text();
item.name    = m_model->item(row, 2)->text();
item.quantity = m_model->item(row, 3)->data(Qt::DisplayRole).toDouble();
item.unit     = m_model->item(row, 4)->text();
item.price    = m_model->item(row, 5)->data(Qt::DisplayRole).toDouble();
item.date     = m_model->item(row, 6)->text();
item.supplier = m_model->item(row, 7)->text();

// Відкриваємо вікно редагування
ItemWindow window(this);

// Передаємо зібраний item у вікно (щоб заповнити поля)
window.setItemData(item);

// Запускаємо вікно і чекаємо результату
if (window.exec() == QDialog::Accepted) {
    //Повідомлення про успішне виконання
    QMessageBox::information(this, "Item added",
                             "Item edited successfully!");
    // Якщо користувач натиснув "Оновити", перезавантажуємо таблицю,
    // щоб побачити зміни
    loadData();
}
}

void MainWindow::on_actionDelete_Selected_Item_triggered()
```

```
{
    // Перевіряємо, чи виділений рядок
    auto selectionModel = ui->tableView->selectionModel();
    auto selectedRows = selectionModel->selectedRows();
    //Повідомлення про пустий рядок
    if (selectedRows.isEmpty()) {
        QMessageBox::warning(this, "Attention", "Please select the product to delete!");
        return;
    }

    // Запитуємо підтвердження
    QMessageBox::StandardButton reply;
    reply = QMessageBox::question(this, "Confirm Delete",
        "Are you sure you want to delete this item?\nThis action cannot
be undone.",
        QMessageBox::Yes | QMessageBox::No);

    if (reply == QMessageBox::No) {
        return;
    }

    // Отримуємо ID з прихованої колонки
    QModelIndex proxyIndex = selectedRows.first();

    // Перетворення індексу
    QModelIndex sourceIndex = m_proxyModel->mapToSource(proxyIndex);
    int row = sourceIndex.row();

    QString id = m_model->item(row, 0)->text();

    // Викликаємо видалення
    if (DatabaseManager::instance().deleteItem(id)) {
        // Якщо успішно — оновлюємо таблицю
        loadData();
        // Повідомлення про успішне виконання
    }
}
```

```
        QMessageBox::information(this, "Success", "Item successfully deleted.");
    } else {
        // Повідомлення про неуспішне виконання
        QMessageBox::critical(this, "Error", "Failed to delete product from database.");
    }
}

void MainWindow::on_actionConnect_to_Database_2_triggered()
{
    ConnectionDialog dialog(this);

    // Відкриваємо вікно і чекаємо, поки користувач натисне "Connect"
    if (dialog.exec() == QDialog::Accepted) {

        // Беремо дані з полів
        QString host = dialog.getHost();
        int port = dialog.getPort();
        QString dbName = dialog.getDbName();

        // Пробуємо підключитися
        bool success = DatabaseManager::instance().connectToDatabase(host, port,
            dbName, "", "");

        // Реагуємо на результат
        if (success) {
            ui->statusbar->showMessage("Connected to: " + host);
            QMessageBox::information(this, "Success", "Successfully connected to the
            database!");

            // Одразу завантажуюємо дані
            loadData();
        } else {
            ui->statusbar->showMessage("Disconnected");
        }
    }
}
```

```
        QMessageBox::critical(this, "Error", "Connection failed. Please check your  
details.");  
    }  
}  
}
```

```
void MainWindow::on_actionConnect_to_Database_triggered()  
{  
    ConnectionDialog dialog(this);  
  
    // Відкриваємо вікно і чекаємо, поки користувач натисне "Connect"  
    if (dialog.exec() == QDialog::Accepted) {  
  
        // Беремо дані з полів  
        QString host = dialog.getHost();  
        int port = dialog.getPort();  
        QString dbName = dialog.getDbName();  
  
        // Пробуємо підключитися  
        bool success = DatabaseManager::instance().connectToDatabase(host, port,  
dbName, "", "");  
  
        // Реагуємо на результат  
        if (success) {  
            ui->statusbar->showMessage("Connected to: " + host);  
            QMessageBox::information(this, "Success", "Successfully connected to the  
database!");  
  
            // Одразу завантажуюмо дані  
            loadData();  
        } else {  
            ui->statusbar->showMessage("Disconnected");  
            QMessageBox::critical(this, "Error", "Connection failed. Please check your  
details.");  
        }  
    }  
}
```

```

    }
}

void MainWindow::on_actionShow_Summary_triggered()
{
    // ЗАХИСТ: Перевірка прав доступу
    // Якщо поточний користувач - Worker, ми його не пускаємо
    if (m_userRole == "Worker") {
        QMessageBox::warning(this, "Access denied",
                               "Only managers have access to financial statistics!");
        return;
    }

    // Отримуємо статистику з бази
    InventoryStats stats = DatabaseManager::instance().getStatistics();

    // Відкриваємо вікно
    StatisticsDialog dialog(this);
    dialog.setStats(stats); // Передаємо підраховані цифри
    dialog.exec();
}

```

Лістингу коду файлу itemwindow.h

```

#ifndef ITEMWINDOW_H
#define ITEMWINDOW_H

#include <QDialog>
#include "databasemanager.h"

namespace Ui {
class ItemWindow;
}

```

Продовження додатка А

```

class ItemWindow : public QDialog
{
    Q_OBJECT

public:
    explicit ItemWindow(QWidget *parent = nullptr);
    ~ItemWindow();
    void setItemData(const Item& item);

private slots:
    void on_okButton_clicked();
    void on_cancelButton_clicked();

private:
    Ui::ItemWindow *ui;
    QString m_currentId;
};

#endif // ITEMWINDOW_H

```

Лістингу коду файлу itemwindow.cpp

```

#include "itemwindow.h"
#include "ui_itemwindow.h"
#include "databasemanager.h"
#include <QMessageBox>

ItemWindow::ItemWindow(QWidget *parent)
    : QDialog(parent)
    , ui(new Ui::ItemWindow)
{
    ui->setupUi(this);
}

```



```
ui->dateEdit->setDate(QDate::currentDate());
if (ui->unitComboBox->count() == 0) {
    ui->unitComboBox->addItem({"pcs", "kg", "l", "pack", "m"});
}
}

ItemWindow::~ItemWindow()
{
    delete ui;
}
//Заповнення полів даними
void ItemWindow::setItemData(const Item &item)
{
    // Запам'ятовуємо ID! Це найголовніше.
    m_currentId = item.id;

    // Заповнюємо поля даними з переданого товару
    ui->articleLineEdit->setText(item.article);
    ui->nameLineEdit->setText(item.name);
    ui->quantityDoubleSpinBox->setValue(item.quantity);
    ui->unitComboBox->setCurrentText(item.unit);
    ui->priceDoubleSpinBox->setValue(item.price);

    // Встановлюємо дату (перетворюємо з рядка)
    ui->dateEdit->setDate(QDate::fromString(item.date, "yyyy-MM-dd"));

    ui->supplierLineEdit->setText(item.supplier);

    // Змінюємо заголовок вікна та текст кнопки
    this->setWindowTitle("Item editing");
    ui->okButton->setText("Update");
}
//Натискання кнопки Ok
void ItemWindow::on_okButton_clicked()
{

```

```

// Валідація: Перевіряємо, чи заповнені обов'язкові поля
if (ui->articleLineEdit->text().isEmpty() || ui->nameLineEdit->text().isEmpty()) {
    QMessageBox::warning(this, "Error", "The fields 'Article' and 'Name' are
required!");
    return;
}

// Збираємо дані з форми у структуру Item
Item item;
item.article = ui->articleLineEdit->text();
item.name = ui->nameLineEdit->text();
item.quantity = ui->quantityDoubleSpinBox->value();
item.unit = ui->unitComboBox->currentText();
item.price = ui->priceDoubleSpinBox->value();
item.date = ui->dateEdit->date().toString("yyyy-MM-dd");
item.supplier = ui->supplierLineEdit->text();

// Відправляємо в базу даних
bool success = false;
if (m_currentId.isEmpty()) {
    // ID немає -> це ДОДАВАННЯ нового
    success = DatabaseManager::instance().addItem(item);
} else {
    // ID є -> це ОНОВЛЕННЯ існуючого
    success = DatabaseManager::instance().updateItem(m_currentId, item);
}
// -----

if (success) {
    this->accept();
} else {
    QMessageBox::critical(this, "Error", "Couldn't save changes!");
}
}

```

```
// Натискання кнопки Cancel
void ItemWindow::on_cancelButton_clicked()
{
    // Просто закриваємо вікно без збереження
    this->reject();
}
```

Лістингу коду файлу connectiondialog.h

```
#ifndef CONNECTIONDIALOG_H
#define CONNECTIONDIALOG_H

#include <QDialog>

namespace Ui {
class ConnectionDialog;
}

class ConnectionDialog : public QDialog
{
    Q_OBJECT

public:
    explicit ConnectionDialog(QWidget *parent = nullptr);
    ~ConnectionDialog();
    QString getHost() const;
    int getPort() const;
    QString getDbName() const;

private slots:
    void on_connectPushButton_clicked();
    void on_cancelPushButton_clicked();
private:
    Ui::ConnectionDialog *ui;};

#endif // CONNECTIONDIALOG_H
```

Лістингу коду файлу connectiondialog.cpp

```
#include "connectiondialog.h"
#include "ui_connectiondialog.h"

ConnectionDialog::ConnectionDialog(QWidget *parent)
    : QDialog(parent)
    , ui(new Ui::ConnectionDialog)
{
    ui->setupUi(this);
    ui->hostLineEdit->setText("localhost");
    ui->portSpinBox->setValue(27017);
    ui->dbNameLineEdit->setText("Warehouse_System");
}

ConnectionDialog::~ConnectionDialog()
{
    delete ui;
}

//Натискання кнопки connect
void ConnectionDialog::on_connectPushButton_clicked()
{
    this->accept(); // Закриває вікно з кодом "Accepted"
}

//Натискання кнопки cancel
void ConnectionDialog::on_cancelPushButton_clicked()
{
    this->reject(); // Закриває вікно з кодом "Rejected"
}

// Геттери
QString ConnectionDialog::getHost() const { return ui->hostLineEdit->text(); }
int ConnectionDialog::getPort() const { return ui->portSpinBox->value(); }
```

```
QString ConnectionDialog::getDbName() const { return ui->dbNameLineEdit->text();  
}
```

Лістингу коду файлу statisticsdialog.h

```
#ifndef STATISTICSDIALOG_H  
#define STATISTICSDIALOG_H  
  
#include <QDialog>  
#include "databasemanager.h"  
  
#include <QtCharts/QChartView>  
#include <QtCharts/QLineSeries>  
#include <QtCharts/QDateTimeAxis>  
#include <QtCharts/QValueAxis>  
  
namespace Ui {  
class StatisticsDialog;  
}  
  
class StatisticsDialog : public QDialog  
{  
    Q_OBJECT  
  
public:  
    explicit StatisticsDialog(QWidget *parent = nullptr);  
    ~StatisticsDialog();  
  
    void setStats(const InventoryStats& stats);  
    void drawChart();  
  
private slots:  
    void on_closePushButton_clicked();  
  
private:
```

```
    Ui::StatisticsDialog *ui;  
};  
  
#endif // STATISTICSDIALOG_H
```

Лістингу коду файлу statisticsdialog.cpp

```
#include "statisticsdialog.h"  
#include "ui_statisticsdialog.h"  
#include <QMap>  
#include <QDateTime>  
  
StatisticsDialog::StatisticsDialog(QWidget *parent)  
    : QDialog(parent)  
    , ui(new Ui::StatisticsDialog)  
{  
    ui->setupUi(this);  
    this->setWindowTitle("Warehouse statistics");  
    this->resize(800, 600);  
}  
  
StatisticsDialog::~StatisticsDialog()  
{  
    delete ui;  
}  
  
void StatisticsDialog::setStats(const InventoryStats &stats)  
{  
    // Виводимо дані в лейбли  
    ui->positionsLabel->setText(QString::number(stats.totalPositions));  
    ui->quantityLabel->setText(QString::number(stats.totalQuantity, 'f', 2));  
  
    // Форматуємо гроші  
    ui->valueLabel->setText(QString::number(stats.totalValue, 'f', 2) + " UAH");  
    drawChart();  
}
```

```
}  
//Малюємо chart  
void StatisticsDialog::drawChart()  
{  
    // Отримуємо всі товари  
    QVector<Item> items = DatabaseManager::instance().getAllItems();  
  
    // Групуємо дані: Дата -> Сумарна кількість  
    // QMap автоматично сортує за датою (ключем)  
    QMap<QDateTime, double> timeSeries;  
  
    for (const Item& item : items) {  
        // Перетворюємо рядок "2025-11-21" у QDateTime  
        QDate date = QDate::fromString(item.date, "yyyy-MM-dd");  
        QDateTime dt(date.startOfDay()); // Початок дня  
  
        if (timeSeries.contains(dt)) {  
            timeSeries[dt] += item.quantity; // Додаємо до існуючої дати  
        } else {  
            timeSeries[dt] = item.quantity; // Нова дата  
        }  
    }  
  
    // Створюємо серію даних для графіка  
    QLineSeries *series = new QLineSeries();  
    series->setName("Revenue dynamics");  
  
    // Знаходимо мін/макс для осей  
    double maxQty = 0;  
    for (auto it = timeSeries.begin(); it != timeSeries.end(); ++it) {  
        // Графік приймає (Час у мілісекундах, Значення)  
        series->append(it.key().toMsecsSinceEpoch(), it.value());  
  
        if (it.value() > maxQty) maxQty = it.value();  
    }  
}
```

```
// Створюємо сам графік
QChart *chart = new QChart();
chart->addSeries(series);
chart->setTitle("Number of items by day");
chart->setAnimationOptions(QChart::SeriesAnimations); // Красива анімація
ПОЯВИ
```

```
// Налаштовуємо вісь X (Час)
QDateTimeAxis *axisX = new QDateTimeAxis;
axisX->setTickCount(10); // Скільки дат показувати
axisX->setFormat("dd.MM"); // Формат підпису: "21.11"
axisX->setTitleText("Date");
chart->addAxis(axisX, Qt::AlignBottom);
series->attachAxis(axisX);
```

```
// Налаштовуємо вісь Y (Кількість)
QValueAxis *axisY = new QValueAxis;
axisY->setLabelFormat("%i"); // Цілі числа
axisY->setTitleText("Quantity (pcs)");
axisY->setRange(0, maxQty * 1.1); // Трохи вище максимуму, щоб було гарно
chart->addAxis(axisY, Qt::AlignLeft);
series->attachAxis(axisY);
```

```
// Відображаємо
QChartView *chartView = new QChartView(chart);
chartView->setRenderHint(QPainter::Antialiasing); // Згладжування ліній
```

```
// Додаємо графік у наш layout, який ми створили в Designer
ui->chartLayout->addWidget(chartView);
```

```
}
```

```
void StatisticsDialog::on_closePushButton_clicked()
{
    this->close();
}
```


Лістингу коду файлу databasemanager.h

```
#ifndef DATABASEMANAGER_H
#define DATABASEMANAGER_H

#include <QString>
#include <QVector>
#include <optional>

#include <mongocxx/client.hpp>
#include <mongocxx/instance.hpp>
#include <mongocxx/uri.hpp>
#include <mongocxx/client.hpp>
#include <mongocxx/instance.hpp>
#include <bsoncxx/json.hpp>
#include <bsoncxx/builder/basic/document.hpp>
#include <bsoncxx/builder/basic/kvp.hpp>

// Проста структура для користувача
struct User {
    QString username;
    QString password; // У реальності тут має бути хеш
    QString role;    // "Manager" або "Worker"
};

struct Item {
    QString id;      // Унікальний ID з Mongo (OID)
    QString article; // Артикул
    QString name;    // Назва
    double quantity; // Кількість
    QString unit;    // Одиниця виміру
    double price;    // Ціна
    QString date;    // Дата
    QString supplier; // Постачальник
};
```

```
struct InventoryStats {  
    int totalPositions;    // Кількість унікальних найменувань  
    double totalQuantity; // Загальна кількість штук (сума всіх quantity)  
    double totalValue;    // Загальна вартість складу (сума quantity * price)  
};  
  
class DatabaseManager  
{  
public:  
    // Патерн Singleton - щоб у нас був один менеджер на всю програму  
    static DatabaseManager& instance();  
  
    // Підключення до бази  
    bool connectToDatabase(const QString& host, int port, const QString& dbName,  
const QString& username, const QString& password);  
    void disconnect();  
    bool isConnected() const;  
  
    // Методи для вікна логіну  
    // Повертає роль користувача, якщо логін/пароль вірні, або std::nullopt  
    std::optional<QString> authenticateUser(const QString& username, const  
QString& password);  
  
    QVector<Item> getAllItems();  
    bool addItem(const Item& item);  
    bool updateItem(const QString& id, const Item& item);  
    bool deleteItem(const QString& id);  
  
    InventoryStats getStatistics();  
  
private:  
    DatabaseManager(); // Приватний конструктор  
    ~DatabaseManager() = default;  
  
    // Внутрішні об'єкти MongoDB
```

```

// instance має бути створений один раз
std::unique_ptr<mongocxx::instance> m_instance;
std::unique_ptr<mongocxx::client> m_client;
mongocxx::database m_db;
bool m_connected = false;
};

#endif // DATABASEMANAGER_H

```

Лістингу коду файлу databasemanager.cpp

```

#include "databasemanager.h"
#include <QDebug>

#include <bsoncxx/json.hpp>
#include <bsoncxx/builder/basic/document.hpp>
#include <bsoncxx/builder/basic/kvp.hpp>

using bsoncxx::builder::basic::kvp;
using bsoncxx::builder::basic::make_document;

DatabaseManager& DatabaseManager::instance()
{
    static DatabaseManager instance;
    return instance;
}

DatabaseManager::DatabaseManager()
{
    // Ініціалізація драйвера
    m_instance = std::make_unique<mongocxx::instance>();
}

// Функція підключення до бази даних
bool DatabaseManager::connectToDatabase(const QString& host, int port, const
QString& dbName, const QString& username, const QString& password)

```

```
{
    try {
        // Формуємо рядок підключення (URI)
        // Формат: mongodb://user:password@host:port
        QString uriString = "mongodb://";

        if (!username.isEmpty() && !password.isEmpty()) {
            uriString += username + ":" + password + "@";
        }

        uriString += host + ":" + QString::number(port);

        // Створюємо URI та клієнта
        mongocxx::uri uri(uriString.toStdString());
        m_client = std::make_unique<mongocxx::client>(uri);

        // Обираємо базу даних
        m_db = (*m_client)[dbName.toStdString()];

        // Перевірка зв'язку (Ping)
        auto adminDb = (*m_client)["admin"];
        adminDb.run_command(make_document(kvp("ping", 1)));

        qDebug() << "Connected to:" << uriString << " DB:" << dbName;
        m_connected = true;
        return true;

    } catch (const std::exception& e) {
        qCritical() << "Connection error:" << e.what();
        m_connected = false;
        return false;
    }
}

//Функція відключення
void DatabaseManager::disconnect()
```

```

{
    // У Mongo драйвері немає явного "disconnect",
    // тому можемо просто скинути клієнта
    m_client.reset();
    m_connected = false;
    qDebug() << "Disconnected.";
}
//Геттер підключення
bool DatabaseManager::isConnected() const
{
    return m_connected;
}

std::optional<QString> DatabaseManager::authenticateUser(const QString
&username, const QString &password)
{
    try {
        auto collection = m_db["Users"];

        auto result = collection.find_one(make_document(
            kvp("username", username.toStdString()),
            kvp("password", password.toStdString())
        ));

        if (result) {
            auto doc = result->view();
            std::string role = std::string(doc["role"].get_string().value);
            return QString::fromStdString(role);
        }
    } catch (const std::exception& e) {
        return QString::fromStdString("Authentication failed");
        qDebug() << "Authentication error:" << e.what();
    }
}

```

```

    return std::nullopt;
}
//Отримання всіх окументів з бази даних
QVector<Item> DatabaseManager::getAllItems()
{
    QVector<Item> items;

    try {
        auto collection = m_db["Data"];

        // Знайти всі документи (порожній фільтр {})
        auto cursor = collection.find({});

        for (auto&& doc : cursor) {
            Item item;

            // Отримуємо ID (перетворюємо OID в рядок)
            item.id = QString::fromStdString(doc["_id"].get_oid().value.to_string());

            // Отримуємо рядкові поля
            if (doc["article"]) item.article =
QString::fromStdString(std::string(doc["article"].get_string().value));
            if (doc["name"]) item.name =
QString::fromStdString(std::string(doc["name"].get_string().value));
            if (doc["unit"]) item.unit =
QString::fromStdString(std::string(doc["unit"].get_string().value));
            if (doc["date"]) item.date =
QString::fromStdString(std::string(doc["date"].get_string().value));
            if (doc["supplier"]) item.supplier =
QString::fromStdString(std::string(doc["supplier"].get_string().value));

            // Отримуємо числа (double)
            if (doc["quantity"]) item.quantity = doc["quantity"].get_double().value;
            if (doc["price"]) item.price = doc["price"].get_double().value;
        }
    }
}

```

```
        items.append(item);
    }
} catch (const std::exception& e) {
    qCritical() << "Error fetching items:" << e.what();
}

return items;
}
//Додання предмету до бази даних
bool DatabaseManager::addItem(const Item &item)
{
    try {
        // Обираємо колекцію з товарами
        auto collection = m_db["Data"];

        // Створюємо документ (BSON)
        auto doc_value = make_document(
            kvp("article", item.article.toStdString()),
            kvp("name", item.name.toStdString()),
            kvp("quantity", item.quantity),
            kvp("unit", item.unit.toStdString()),
            kvp("price", item.price),
            kvp("date", item.date.toStdString()),
            kvp("supplier", item.supplier.toStdString())
        );

        // Відправляємо в базу
        // insert_one повертає результат, який містить ID нового запису
        auto result = collection.insert_one(doc_value.view());

        // Перевіряємо результат
        if (result) {
            // Для налагодження виведемо ID створеного запису
            qDebug() << "Successfully added item. New ID:"
```

```

<<QString::fromStdString(result->inserted_id().get_oid().value.to_string());
    return true;
}

} catch (const std::exception& e) {
    qCritical() << "Error adding item to MongoDB:" << e.what();
}

return false; // Якщо сталася помилка
}
//Оновлення предмету в базі даних
bool DatabaseManager::updateItem(const QString &id, const Item &item)
{
    try {
        auto collection = m_db["Data"];

        // Створюємо фільтр (шукаємо по _id)
        bsoncxx::oid oid(id.toStdString());
        auto filter = make_document(kvp("_id", oid));

        // Створюємо команду оновлення ($set)
        auto update_doc = make_document(kvp("$set", make_document(
            kvp("article", item.article.toStdString()),
            kvp("name", item.name.toStdString()),
            kvp("quantity", item.quantity),
            kvp("unit", item.unit.toStdString()),
            kvp("price", item.price),
            kvp("date", item.date.toStdString()),
            kvp("supplier", item.supplier.toStdString())
        ))));

        // Виконуємо оновлення
        auto result = collection.update_one(filter.view(), update_doc.view());

        return result && result->modified_count() > 0;
    }
}

```



```
} catch (const std::exception& e) {
    qCritical() << "Error updating item:" << e.what();
    return false;
}
}
//Видалення предмету
bool DatabaseManager::deleteItem(const QString &id)
{
    try {
        auto collection = m_db["Data"];

        // Перетворюємо рядок ID у формат MongoDB ObjectId
        bsoncxx::oid oid(id.toStdString());

        // Створюємо фільтр: видалити документ, у якого "_id" == oid
        auto filter = make_document(kvp("_id", oid));

        // Виконуємо видалення
        auto result = collection.delete_one(filter.view());

        // Перевіряємо, чи було щось видалено (deleted_count > 0)
        if (result && result->deleted_count() > 0) {
            qDebug() << "Item deleted successfully. ID:" << id;
            return true;
        } else {
            qWarning() << "Item not found or not deleted. ID:" << id;
            return false;
        }
    }

    catch (const std::exception& e) {
        qCritical() << "Error deleting item:" << e.what();
        return false;
    }
}
```

```
InventoryStats DatabaseManager::getStatistics()
{
    InventoryStats stats;
    stats.totalPositions = 0;
    stats.totalQuantity = 0.0;
    stats.totalValue = 0.0;

    // Отримуємо всі товари (використовуємо вже існуючий метод)
    QVector<Item> items = getAllItems();

    stats.totalPositions = items.size();

    // Проходимося по списку і сумуємо
    for (const Item& item : items) {
        stats.totalQuantity += item.quantity;
        stats.totalValue += (item.quantity * item.price);
    }

    return stats;
}
```